Análise do Impacto de um Pré-Processamento Simétrico $O(n)$ na Redução de Inversões e Eficiência de Algoritmos de Ordenação

Samuel da Penha Nascimento [1], **Francisco das Chagas Rocha** [1]

[1]Coordenação do Curso de Sistemas de Computação, Universidade Estadual do Piauí
Campus Prof. Alexandre Alves de Oliveira
Parnaíba, Piauí - Brasil

samuelpnascimentodev@gmail.com, rocha@phb.uespi.br

Abstract. *Quadratic algorithms are useful in resource-constrained systems, yet sensitive to input inversions. However, the efficacy of reducing these inversions via preprocessing across different algorithm classes remains poorly explored. This paper evaluates an $O(n)$ symmetric inversion preprocessing on adaptive and non-adaptive algorithms. To this end, eight methods were compared across ten input topologies with vectors of up to 10^7 elements. In unaligned inputs, preprocessing removed 40%–62% of inversions, with gains of 14%–60% in adaptive quadratic methods at n=10,000. Near-total elimination and speedups of ×386–2,280 occurred only in the Inverted and Zigzag topologies and do not generalize: in Inverted, a trivial descending-run detection with reversal alone matches the hybrid, showing that the extreme follows from input geometry rather than from an advantage of the preprocessing; in Zigzag, the outcome is reported only as an empirical effect of the full pass, without closed theoretical characterization. In $O(n \log n)$ methods the effects were mixed and confined to aligned topologies, with systematic degradation of the evaluated Quicksort and on the remaining inputs; no hybrid outperformed the standard sorter on random inputs.*

Resumo. *Algoritmos quadráticos são úteis em sistemas com recursos restritos, porém sensíveis às inversões na entrada. Todavia, a eficácia de reduzir essas inversões via pré-processamento em diferentes classes de algoritmos é pouco explorada. Este artigo avalia um pré-processamento simétrico de inversões, de complexidade $O(n)$, em algoritmos adaptativos e não adaptativos. Para tanto, compararam-se oito métodos em dez topologias de entrada com vetores de até 10^7 elementos. Em entradas não alinhadas, o pré-processamento removeu de 40% a 62% das inversões, com ganhos de 14% a 60% em métodos quadráticos adaptativos em n=10.000. A eliminação quase total e os* speedups *de ×386–2.280 ocorreram somente nas topologias Invertido e Zigue-zague e não se generalizam: no Invertido, uma detecção trivial de run descendente com reversão sozinha iguala o híbrido, indicando que o extremo decorre da geometria da entrada e não de vantagem do pré-processamento; no Zigue-zague, o desfecho é relatado apenas como efeito empírico da passada completa, sem caracterização teórica fechada. Nos métodos $O(n \log n)$, os efeitos foram mistos e restritos a topologias alinhadas,*

com degradação sistemática do Quicksort avaliado e nas demais entradas; nenhum híbrido superou o ordenador padrão em entradas aleatórias.

1. Introdução

A ordenação de dados é uma operação fundamental na Ciência da Computação [Knuth 1973, Sedgewick and Wayne 2011]. Embora métodos $O(n \log n)$, como Merge Sort e Quicksort, sejam preferíveis para grandes volumes, algoritmos quadráticos $O(n^2)$ — como Insertion Sort, Selection Sort e Bubble Sort — permanecem relevantes em cenários específicos, pela simplicidade estrutural, baixo consumo de memória (*in-place*) e eficiência em dados pequenos ou parcialmente ordenados [Cormen et al. 2022, Ziviani 2021].

A eficiência prática de um algoritmo de ordenação não depende apenas de sua complexidade assintótica, mas também das propriedades da sequência de entrada [Estivill-Castro and Wood 1992]. A inversão, definida como um par (i, j) com $i < j$ e $A[i] > A[j]$, é uma medida quantitativa do grau de desordem de um arranjo [Knuth 1973, Mannila 1985]. Sequências com maior quantidade de inversões exigem mais esforço para serem ordenadas por operações locais [Mannila 1985], o que influencia o custo de execução de algoritmos sensíveis a essa propriedade [Estivill-Castro and Wood 1992].

Nesse cenário, uma etapa preliminar capaz de reduzir a desordem da entrada pode diminuir o trabalho necessário na ordenação subsequente. O objetivo deste trabalho é propor, implementar e avaliar um algoritmo simétrico de pré-processamento de baixo custo $O(n)$, projetado para reduzir o número de inversões de um vetor de entrada. Busca-se verificar experimentalmente se a abordagem híbrida apresenta maior eficiência temporal que a aplicação direta dos algoritmos convencionais, identificando as condições nas quais $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$, onde C_{pre} é o custo do pré-processamento e C_{sort} o da ordenação subsequente. A comparação inclui os ordenadores gerais mais rápidos disponíveis (`sort_unstable`/`sort`) e uma detecção de runs $O(n)$ com reversão; uma teoria fechada do valor esperado de redução permanece como trabalho futuro.

A relevância deste estudo está na investigação de uma estratégia capaz de melhorar o comportamento prático de algoritmos de ordenação sem modificar sua estrutura fundamental ou introduzir elevado custo computacional adicional [Estivill-Castro and Wood 1992, Hwang et al. 2000]. A redução do grau de desordem antes da ordenação principal pode diminuir o número de operações em entradas desfavoráveis, especialmente em métodos sensíveis à quantidade de inversões, e contribui para a análise de algoritmos adaptativos ao investigar as condições em que o custo do pré-processamento é compensado [Mannila 1985, Hwang et al. 2000].

Para avaliar a eficácia do pré-processamento proposto, realizou-se um estudo experimental com implementação em Rust, submetido a benchmarks automatizados contra algoritmos de ordenação consolidados na literatura e ordenadores de propósito geral. A avaliação abrangeu vetores sintéticos sob variados níveis de inversão — do pior caso a topologias adversariais — bem como dados de um cenário real.

Este artigo está estruturado em seis seções, além desta introdução. A Seção

2 apresenta o Referencial Teórico. A Seção 3 descreve os Métodos e Técnicas da Pesquisa, a formalização do algoritmo de pré-processamento e os procedimentos experimentais. A Seção 4 apresenta e discute os resultados obtidos. A Seção 5 apresenta as Considerações Finais. Por fim, a Seção 6 declara o uso de Inteligência Artificial e a Seção 7 disponibiliza o repositório da pesquisa.

2. Referencial Teórico

Nesta seção apresentam-se os fundamentos teóricos necessários: o conceito de inversão como medida de desordem, os princípios da ordenação adaptativa, as características dos algoritmos avaliados, o pré-processamento simétrico e os trabalhos relacionados que situam a contribuição desta pesquisa no contexto da literatura.

2.1. Inversões, Ordenação Adaptativa e Algoritmos Avaliados

A eficiência dos algoritmos de ordenação tradicionais depende da disposição inicial dos elementos [Cormen et al. 2022, Estivill-Castro and Wood 1992]. Uma inversão é um par (i, j) com $i < j$ e $a[i] > a[j]$ [Knuth 1973, Sedgewick and Wayne 2011]; o número total de inversões I mede o grau de desordem da entrada e seu esforço de ordenação por operações locais [Mannila 1985]. A relação entre I e a complexidade de execução origina a propriedade de *adaptabilidade* [Estivill-Castro and Wood 1992]: um algoritmo adaptativo melhora sua complexidade temporal na presença de ordenamento prévio parcial. O *Insertion Sort* é um caso clássico: consome $O(n + I)$ operações, atingindo $O(n)$ quando $I = 0$ e degradando para $O(n^2)$ no pior caso [Cormen et al. 2022, Knuth 1973].

No grupo dos algoritmos quadráticos foram avaliados o *Bubble Sort*, o *Insertion Sort* e o *Selection Sort* [Cormen et al. 2022]. Os dois primeiros são adaptativos, reduzindo o número de operações conforme o grau de ordenamento prévio [Estivill-Castro and Wood 1992]. O *Selection Sort* é não adaptativo, com número fixo de comparações independentemente da entrada [Estivill-Castro and Wood 1992], servindo como contraponto aos sensíveis ao pré-processamento.

Entre os quasilineares, avaliaram-se o *Merge Sort* ($O(n \log n)$) e o *Quicksort* (médio $O(n \log n)$, pior caso $O(n^2)$) [Cormen et al. 2022]. Essa composição — adaptativos quadráticos, não adaptativo quadrático, e métodos quasilineares — permite verificar tanto a redução de inversões como a conversão dessa redução em ganhos de desempenho para algoritmos com distintos graus de dependência ordem-inicial.

2.2. Pré-Processamento Simétrico

O conceito de pré-processamento, na análise de algoritmos, refere-se a uma etapa preliminar de transformação dos dados de entrada realizada antes da execução do algoritmo principal [Cormen et al. 2022]. Para ser viável em problemas de ordenação, seu custo temporal deve ser estritamente dominado pela complexidade do algoritmo subsequente [Cormen et al. 2022, Sedgewick and Wayne 2011]. Um pré-processamento ideal apresenta complexidade linear $O(n)$, garantindo que o *overhead* introduzido seja assintoticamente desprezível.

Esta pesquisa denomina a etapa de *simétrica* por aplicar varredura bidirecional e espelhada a partir das extremidades do vetor, reestruturando a disposição dos

elementos em um único passe de complexidade $O(n)$. O objetivo é reduzir o número total de inversões I da entrada ao mover elementos discrepantes para posições mais próximas de seus destinos finais. Ao fornecer uma estrutura parcialmente reordenada para os algoritmos subsequentes, busca-se mitigar comparações e trocas tanto em métodos adaptativos quanto em não adaptativos.

2.3. Trabalhos Relacionados

A literatura apresenta abordagens para caracterizar o grau de ordenação de sequências [Mannila 1985, Hwang et al. 2000]. Mannila [Mannila 1985] definiu conceitualmente a pré-ordenação e estabeleceu medidas formais para quantificar a desordem, tomando o número de inversões como métrica central e demonstrando formalmente que o *Insertion Sort* é um algoritmo ótimo para três medidas naturais de pré-ordenação, estabelecendo os fundamentos para a análise de algoritmos adaptativos. Hwang, Yang e Yeh [Hwang et al. 2000] abordam algoritmos de *presorting* sob a perspectiva do caso médio, conforme o título e o registro bibliográfico do artigo (Theoretical Computer Science, v. 242, pp. 29–40). O conteúdo analítico exato daquele artigo não pôde ser verificado neste trabalho (texto completo indisponível; apenas metadados via Crossref/OpenAlex foram consultados), de modo que nenhuma expressão, valor esperado ou conclusão específica é aqui atribuída a esses autores e nenhuma transferência de resultados é reivindicada para o algoritmo avaliado. Em particular, a redução de $45,19\%$ medida no Aleatório (Tabela 3, n=10.000) é resultado empírico próprio, obtido nos vetores do *pool* pareado deste estudo, e não uma justificativa empírica proveniente de Hwang et al. A contribuição original do presente trabalho em relação a Mannila (medidas e otimalidade adaptativa) e a Hwang et al. (análise de caso médio de esquemas de *presorting*) consiste em propor, implementar e medir experimentalmente uma heurística simétrica específica de passada única $O(n)$, além de derivar, na Seção 3.4, um termo de primeira ordem próprio para a componente de trocas espelhadas considerada isoladamente.

Estudos recentes aprofundam a interface entre adaptatividade e eficiência no hardware. Edelkamp e Weiss [Edelkamp and Weiß 2016] desenvolveram o algoritmo *BlockQuicksort* para mitigar a degradação por falhas de predição de desvio no Quicksort tradicional. Auger et al. [Auger et al. 2018] analisaram o pior caso do *Timsort*, demonstrando eficiência assintótica e garantias de estabilidade em bibliotecas padrão de linguagens modernas. Diferenciando-se dessas abordagens, este trabalho objetiva avaliar uma heurística de pré-processamento simétrico projetada para a mitigação direta de inversões em tempo linear $O(n)$, investigando implicações teóricas e práticas em dez topologias de entrada e delimitando o escopo de aplicabilidade ao isolar cenários em que o ganho pela redução de inversões supera o custo linear da etapa preliminar.

3. Métodos e Técnicas da Pesquisa

A presente pesquisa é classificada, quanto à sua natureza, como aplicada, pois gera conhecimentos voltados à aplicação prática e dirigidos à solução de problemas específicos [Gil 2019]. Quanto à abordagem, é quantitativa, fundamentando-se na coleta de dados baseada em medição numérica e análise estatística para testar hipóteses e estabelecer padrões [Sampieri et al. 2021]. Quanto aos objetivos, é

explicativa, com preocupação central em identificar os fatores que determinam ou contribuem para a ocorrência dos fenômenos [Gil 2019]. Quanto aos procedimentos, é experimental, manipulando de forma controlada a topologia e o tamanho das entradas (variáveis independentes) e observando o tempo de execução e as inversões (variáveis dependentes), sob semente fixa, geração pareada de vetores e ambiente computacional padronizado [Gil 2019].

3.1. Configuração

Os experimentos foram realizados em ambiente controlado. O plano de energia foi mantido em alto desempenho e os processos em segundo plano minimizados. O código foi compilado com `opt-level = 3`. A geração de dados empregou semente fixa e entradas pareadas para garantir reprodutibilidade, com 50 amostras por combinação de topologia e tamanho, e 5 *pools* independentes para análise de estabilidade.

Tabela 1. Ambiente experimental e ferramentas utilizadas.

Componente	Configuração / Especificação
Sistema operacional	Windows 11 Pro (build 10.0.26200)
Processador	AMD Ryzen 7 8700G (8 núcleos / 16 threads)
Memória	16 GB DDR5 5600MT/s (dual channel)
Linguagem	Rust 1.96.0 (perfil *release*)
Ferramentas de Medição	Criterion 0.5 e Ferramenta CLI do Projeto
Geração de dados	Seeds 42–46 (`ChaCha8Rng`), entradas pareadas

Para avaliar a resiliência e a adaptabilidade dos algoritmos sob diferentes graus de entropia, foram definidos dez padrões estruturais de vetores de entrada (Tabela 2). A geração determinística utilizou sementes pareadas entre as abordagens: para cada combinação de topologia e tamanho, foram gerados 50 vetores com a semente $42 \oplus n \oplus \text{tipo}$, utilizando `ChaCha8Rng`. Os mesmos vetores foram compartilhados por todos os algoritmos e variantes, garantindo a comparabilidade das medições. Nos padrões determinísticos (Invertido, Zigue-zague e Tartarugas), as réplicas do *pool* são idênticas; nos estocásticos (Aleatório, Duplicados e Quase ordenado), variam entre réplicas, mas permanecem pareadas entre algoritmos. Por fim, a matriz experimental foi replicada em cinco *pools* independentes (seeds 42–46) para avaliar a estabilidade dos resultados (Seção 3.3).

Os algoritmos quasilineares (*Merge Sort* e *Quicksort*) e os baselines da biblioteca padrão foram medidos em $n \in \{16, \ldots, 10.000.000\}$. Para os algoritmos quadráticos (*Insertion Sort*, *Bubble Sort*, *Selection Sort* e *DescReverse*), as medições foram restritas a $n \leq 100.000$: em $n = 10^6$, cada execução exigiria tempo da ordem de minutos, inviabilizando a coleta sistemática de amostras significativas. Essa restrição é documentada e não compromete a comparação, pois a ordem de grandeza dos ganhos já se estabiliza em $n = 100.000$. Os pontos em n=10^7 correspondem a ≈40 MB por vetor `i32`; a topologia Real é limitada ao tamanho nativo do asset (N=477.104, sem *tiling*).

O custo isolado do pré-processamento (C_{pre}) foi adicionalmente caracterizado em $n \in \{1.000, 5.000, 10.000, 20.000, 100.000, 1.000.000\}$ (Tabela 7 e Figura 2) para

melhor evidenciar a linearidade; o ponto n=20.000 foi coletado exclusivamente para essa curva de custo via ferramenta CLI (`seed` 42), sob mesmo protocolo de geração pareada, e não faz parte da matriz principal de comparação de tempos de ordenação.

Tabela 2. Conjuntos de dados e caracterização das topologias de entrada.

Topologia	Descrição do Padrão Estrutural
Aleatório	permutação uniforme de $[0, n-1]$ via `shuffle` com `ChaCha8Rng`
Tartarugas	$a[i] = i + n$ se $i < n/2$, senão $a[i] = i \bmod 10$
Zigue-zague	$a[i] = i$ se i par, senão $a[i] = n - i$; determinístico
Quase ordenado	$[0, n-1]$ ordenado com $n/100$ trocas de pares aleatórios
Duplicados	$a[i] \sim \mathcal{U}\{0, 1, 2\}$ i.i.d.
Invertido	$a[i] = n - i$ (estritamente decrescente); determinístico
Serra	$a[i] = i \bmod 16$
Tubo	$a[i] = i$ se $i < n/2$, senão $a[i] = n - 1 - i$
Runs	4 blocos invertidos disjuntos nos quartis do vetor
Real	prefixo de n bytes de asset congelado (N=477.104)

3.2. Especificação e Implementação dos Algoritmos Avaliados

Para assegurar a validade interna dos experimentos, a estabilidade das medições de tempo e a reprodutibilidade dos resultados, todos os algoritmos de ordenação foram implementados na linguagem Rust (versão 1.96.0), operando diretamente sobre fatias mutáveis de inteiros de 32 bits (`&mut [i32]`). As implementações evitaram dependências externas de abstrações de alto nível, padronizando a manipulação de memória e adotando otimizações idiomáticas no nível de instrução.

3.2.1. Algoritmos Quadráticos

- **Bubble Sort:** Instanciado em sua versão adaptativa otimizada. O algoritmo realiza varreduras sucessivas reduzindo o limite superior a cada iteração ($n - 1 - i$). A adaptabilidade é garantida por uma flag booleana de controle (`trocou`) que monitora a ocorrência de trocas em cada passada; caso uma varredura seja concluída sem nenhuma permuta de elementos, a execução é interrompida precocemente, garantindo complexidade linear $O(n)$ em vetores previamente ordenados.
- **Insertion Sort:** Implementado conforme a mecânica tradicional de inserção direta por deslocamento. Para cada elemento da sub-fatia não ordenada, armazena-se o valor corrente em uma variável temporária (`valor_atual`) e promovem-se deslocamentos à direita dos elementos antecedentes estritamente maiores. A busca e o deslocamento encerram-se assim que a posição correta é encontrada ou ao atingir o limite esquerdo ($j = 0$), momento em que o valor temporário é atribuído. Essa abordagem minimiza o *overhead* de escrita em relação às implementações baseadas em trocas sucessivas (*swaps*).
- **Selection Sort:** Implementado em sua forma clássica não adaptativa e *in-place*. Em cada iteração i do laço externo, o algoritmo varre o segmento não ordenado $[i + 1, n - 1]$ para identificar o índice do menor elemento

(`indice_minimo`). A troca (`swap`) com a posição i é executada condicionalmente apenas se o menor elemento encontrado residir em um índice distinto (`indice_minimo` $\neq i$). O algoritmo mantém um custo de comparações estrito de $O(n^2)$, independentemente da ordenação prévia da entrada.

3.2.2. Algoritmos Quasilineares

- **Merge Sort:** Implementado segundo a estratégia *top-down* (divisão e conquista), com garantia de ordenação estável. Para evitar alocações dinâmicas recorrentes durante a recursão e reduzir interferências nas medições temporais, utiliza um único vetor auxiliar de tamanho n (`vec![0; len]`), alocado na entrada da função e compartilhado entre as chamadas recursivas por referência mutável. Os pontos médios das sub-fatias são calculados por deslocamento de bits à direita ($(\text{fim} - \text{inicio}) \gg 1$), evitando a divisão inteira. Na etapa de intercalação (`intercalar`), os elementos do segmento atual são copiados para o *buffer* auxiliar e reorganizados no vetor original por meio de três índices.
- **Quicksort:** Implementado de forma recursiva utilizando particionamento de três vias (*Dutch National Flag* / algoritmo de Dijkstra) aliado à seleção de pivô por mediana de três. O pivô é escolhido como a mediana entre os elementos nos índices inicial, central e final do subvetor. O particionamento de três vias rearranja o subvetor em três regiões contíguas: elementos menores que o pivô, elementos iguais ao pivô e elementos maiores que o pivô, mantendo índices para delimitar as fronteiras. Essa estratégia confere tempo médio $O(n \log n)$ e robustez a muitas chaves duplicadas (sem garantia de pior caso: a mediana de três não exclui entradas adversariais). A recursão é aplicada apenas às regiões de elementos menores e maiores que o pivô.

3.2.3. Baselines Competitivos

- **`sort_unstable`:** a ordenação instável da biblioteca padrão do Rust, o ordenador geral mais rápido disponível e a referência que qualquer proposta precisa superar.
- **`sort`:** a ordenação estável da biblioteca padrão, incluída como referência de pior caso estável.
- **`desc_reverse`:** verificação $O(n)$ de run estritamente decrescente; se a entrada é estritamente decrescente, ela é revertida in-place e então executa-se o Insertion Sort (caso contrário, Insertion Sort direto). Testa se uma detecção trivial de run explica os ganhos no Invertido.

Em todas as implementações, verificações preliminares de contorno (*early returns*) asseguram que fatias com tamanho $n < 2$ retornem imediatamente sem incorrer em chamadas de função ou alocações desnecessárias.

3.3. Protocolo Experimental e Raciocínio de Cálculo de Inversões

O protocolo de medição empírica foi desenhado para isolar interferências do ambiente e garantir a reprodutibilidade estatística das métricas temporais. A quantificação do

tempo de execução utilizou o arcabouço *Criterion.rs*, que executa ciclos automatizados de aquecimento de memória cache (*warm-up*) e realiza reamostragens estatísticas internas (*bootstrapping* do próprio Criterion) para estimar intervalos de confiança de 95% (`confidence_level=0.95`, `significance_level=0.05`). Esse *bootstrap* interno do Criterion — usado apenas para os ICs das médias/medianas por braço — não se confunde com o *bootstrap* pareado do teste de efeito (abaixo), implementado fora do Criterion no exemplo `efeito_pareado`. Para cada combinação de algoritmo, topologia e tamanho de vetor n, foram calculados o tempo médio, a mediana e os intervalos de confiança de 95%.

A configuração das medições foi ajustada conforme o tamanho e a complexidade das entradas. Aqui, "amostras" designa exclusivamente o parâmetro `sample_size` do Criterion, isto é, o número de medições repetidas que o Criterion coleta por braço (puro, com pré, ou pré isolado) para estimar médias e ICs — não o número de vetores de entrada distintos. Os perfis aplicados no `benchmark_group` (`benches/benchmark.rs`, função `config_mediacao`) são: 50 amostras, aquecimento de 2 s e medição de 5 s por braço na maioria das células; 30 amostras, aquecimento de 1 s e medição de 5 s para os algoritmos quadráticos em $n = 100.000$; 50 amostras, aquecimento de 2 s e medição de 10 s para os métodos quase lineares em $n \geq 10^6$. O braço de custo isolado do pré-processamento (`So_Pre`) usa 50 amostras com 2 s/5 s em todas as células. Ressalta-se que o Criterion pode ampliar o tempo efetivo de medição quando necessário para coletar a quantidade configurada de amostras.

Para garantir uma comparação justa, a avaliação temporal das abordagens com pré-processamento contabilizou o custo combinado das duas etapas, conforme $C_{\text{total}} = C_{\text{pre}} + C_{\text{sort}}$. Dessa forma, a viabilidade prática da abordagem foi avaliada considerando a condição $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$, evitando que o ganho obtido na ordenação fosse analisado isoladamente do custo adicional introduzido pelo pré-processamento.

Para cada par (topologia, tamanho), o *harness* gera um *pool* único de R=50 vetores (`AMOSTRAS_POR_POOL`, `ChaCha8Rng` com semente `seed` $\oplus$ `tamanho` $\oplus$ `tipo`; nos padrões determinísticos as réplicas são idênticas, nos estocásticos variam mas permanecem pareadas entre algoritmos). O mesmo *pool* alimenta os três braços da célula (puro, com pré e pré isolado) e todos os algoritmos. Como o pré-processamento e as ordenações operam *in-place*, nenhum vetor do *pool* é reutilizado entre iterações sem restauração: a cada iteração do Criterion (`iter_batched`), a rotina de *setup* — fora da região cronometrada — clona o próximo vetor do *pool* em rotação (`pool[i].clone()`, índice circular) e entrega o clone à rotina medida. A região cronometrada contém somente a chamada a `BenchmarkServiceBench::medir_*` sobre esse clone: no braço puro, apenas a ordenação (`medir_puro`); no braço com pré, o pré-processamento seguido da ordenação sobre o mesmo clone (`medir_com_pre`, com um único `Instant::now()` cobrindo as duas etapas); no braço isolado, apenas o pré-processamento (`medir_pre`). O custo do clone/da restauração não integra a região cronometrada. Assim, cada braço inicia cada uma de suas iterações a partir de um clone fresco de um vetor do *pool* (rotação circular), de modo que os braços amostram o mesmo *pool* e o vetor original jamais é modificado; o pareamento estrito por vetor (mesmo índice do *pool* nos dois braços, mesma repetição) é garantido apenas

no teste de efeito abaixo, não dentro do Criterion, cujos braços são executados como *benchmarks* separados.

Critério estatístico (inferência pareada, fora do Criterion): para cada célula e cada *pool*, o exemplo `efeito_pareado` (`FP_R`= 50 vetores, `FP_BOOT`= 10.000, cinco *pools* com seeds 42–46) reconstrói o mesmo *pool* do *bench*, clona cada vetor e cronometra os dois braços sobre clones do mesmo vetor (ordem dos braços alternada por repetição para controlar deriva), obtendo um par $(T_{\text{puro}}, T_{\text{pre}})$ por vetor (cada T é a média de repetições internas calibradas para $\approx$50 μs por medição, com o mesmo clone-por-repetição; v. `efeito_pareado.rs`). A estatística de efeito é a média das log-razões por vetor $\overline{\log(T_{\text{pre}}/T_{\text{puro}})}$, com intervalo de confiança percentílico por *bootstrap* de 95% (B=10.000 reamostragens dos R=50 pares, RNG determinística por célula). Aqui, portanto, R=50 é o número de vetores distintos do *pool* (unidades pareadas), ao passo que as "50 amostras" do parágrafo anterior são repetições de medição do Criterion por braço — grandezas distintas, ainda que numericamente coincidentes na maioria das células. Um efeito é relatado como significativo quando o IC pareado exclui o zero, marcado com * nas tabelas completas (Tabela 5 e Apêndice); † marca efeitos significativos *e* estáveis em sinal nos cinco *pools*. Células significativas no pareamento mas instáveis entre *pools* (ex. Quicksort/Quase ordenado em n=10^6) são explicitamente sinalizadas como não robustas. Isso substitui a regra conservadora anterior de não sobreposição de ICs, que a análise pareada supera (notavelmente, Selection/Zigue-zague em n=10.000 é não significativo no pareamento).

Além das métricas temporais, a corretude de cada cenário foi verificada fora da região cronometrada: para todo (algoritmo, topologia, tamanho), confirma-se que a saída é uma permutação ordenada da entrada (comparação com `sort_unstable`) tanto na variante pura quanto na com pré-processamento. O número de inversões de cada vetor do conjunto compartilhado, antes e depois do pré-processamento, é registrado em arquivos auxiliares (*companion*) do harness (`target/criterion/.../_companion/inversoes_{tipo}_{n}.csv`), garantindo que as tabelas de inversões deste artigo derivam exatamente dos mesmos vetores cronometrados.

A quantificação experimental do número exato de inversões I utiliza uma implementação baseada no algoritmo MergeSort modificado, com complexidade temporal $O(n \log n)$ [Cormen et al. 2022]. Durante a etapa de intercalação de dois subvetores ordenados L e R, quando um elemento $R[j]$ é menor que $L[i]$, adiciona-se exatamente $|L| - i$ ao contador de inversões. Essa abordagem elimina o gargalo quadrático da contagem direta $O(n^2)$, permitindo a avaliação de vetores com ordens de grandeza de 10^5 a 10^6 elementos dentro do protocolo experimental.

3.4. Formalização do Pré-Processamento Simétrico

O procedimento de pré-processamento simétrico proposto opera como uma etapa determinística de varredura única sobre o vetor de entrada A de tamanho n. O objetivo desta etapa é reestruturar a disposição dos elementos através de trocas espelhadas e ajustes locais, antecedendo a execução dos algoritmos de ordenação avaliados. A estrutura procedural e as regras de controle do algoritmo estão formalizadas no Algoritmo 1.

Algoritmo 1: Algoritmo de Pré-Processamento Simétrico

Input: Vetor A de tamanho n
Output: Vetor A parcialmente reorganizado
$n \leftarrow$ tamanho de A;
if $n < 2$ **then**
  return;
end
$ultimo \leftarrow n - 1$;
$meio \leftarrow \lfloor n/2 \rfloor$;
for $i \leftarrow 0$ **to** $meio - 1$ **do**
  $j \leftarrow ultimo - i$;
  if $A[i] > A[j]$ **then**
    trocar$(A[i], A[j])$;
  end
  if $i + 1 < meio$ **then**
    if $A[i] > A[i + 1]$ **then**
      trocar$(A[i], A[i + 1])$;
    end
  end
  if $j - 1 > meio$ **then**
    if $A[j] < A[j - 1]$ **then**
      trocar$(A[j], A[j - 1])$;
    end
  end
end

O algoritmo itera sobre a primeira metade do vetor, varrendo os índices $i \in [0, \lfloor n/2 \rfloor - 1]$. A cada iteração, calcula-se o índice simétrico correspondente no extremo oposto do vetor, dado por $j = (n - 1) - i$. A execução do laço é composta por três verificações condicionais estruturadas:

1. **Troca Simétrica Extrema:** Avalia-se a relação de ordem entre as extremidades opostas. Se $A[i] > A[j]$, efetua-se a troca direta dos elementos, posicionando o menor elemento no segmento esquerdo e o maior no segmento direito.
2. **Ajuste Adjacente Esquerdo:** Para os índices em que $(i + 1) < \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade esquerda. Se $A[i] > A[i+1]$, realiza-se a troca entre os elementos adjacentes.
3. **Ajuste Adjacente Direito:** Para os índices em que $(j-1) > \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade direita. Se $A[j] < A[j - 1]$, executa-se a troca entre os elementos adjacentes.

As condições de contorno garantem que as trocas adjacentes operem estritamente dentro de seus respectivos subdomínios (esquerdo e direito), sem ultrapassar o ponto médio do vetor. Em termos de complexidade computacional, o algoritmo executa exatamente $\lfloor n/2 \rfloor$ iterações. Em cada passo do laço, realiza-se um número constante de comparações (no máximo 3) e trocas (no máximo 3), resultando em uma complexidade temporal de pior caso $T(n) = O(n)$. Como a reestruturação dos

dados é realizada *in-place* no próprio vetor A, a complexidade espacial auxiliar é estritamente $O(1)$.

Uma propriedade útil de monotonicidade é que trocar um par espelhado invertido nunca aumenta I: remove-se a inversão do par (i, j), e cada posição intermediária k com $a_j < a_k < a_i$ passa de duas inversões para zero (redução líquida de 2), enquanto as demais mantêm sua contribuição; logo a passada espelhada isolada é um passo de descida em I. O termo de primeira ordem a seguir é derivação própria deste trabalho, e não resultado atribuído a Hwang et al.: para entrada aleatória uniforme (permutação uniforme) e considerando somente a componente de trocas espelhadas executada isoladamente (sem os ajustes adjacentes do Algoritmo 1), condicionando-se em uma troca espelhada em (i, j) (que ocorre com probabilidade $1/2$), a troca remove a inversão do par (i, j), e cada k interior troca os pares (i, k) e (k, j) de $X = \mathbf{1}[a_i > a_k] + \mathbf{1}[a_k > a_j]$ para $2 - X$, com redução líquida esperada de $2/3$ por k. A tomada de esperança sobre os $n/2$ pares espelhados fornece $\mathbb{E}[\Delta I_{\text{esp}}] = n^2/12 + O(n)$, cerca de um terço da base $\mathbb{E}[I] = n^2/4$. Esse termo vale somente para a componente de trocas espelhadas executada isoladamente: não se estende à passada completa, na qual cada ajuste adjacente remove no máximo uma inversão (contribuição total $O(n)$). Os 45,19% medidos no Aleatório (Tabela 3, n=10.000) são resultado empírico próprio deste trabalho, obtido nos mesmos vetores pareados cronometrados, e excedem esse termo em $\Theta(n^2)$. A explicação desse excedente pela interação sequencial entre trocas espelhadas e ajustes dentro da mesma passada é hipótese interpretativa dos autores, sem decomposição por componente reportada neste artigo e sem forma fechada demonstrada; uma forma fechada para o algoritmo completo permanece como trabalho futuro.

Para métodos adaptativos com custo de $\approx c$ por inversão ($c \approx 0{,}21$ ns medidos, Seção 4.2), o pré-processamento compensa se e somente se $C_{\text{pre}} < c\,\Delta I$. Com $C_{\text{pre}} \approx \alpha n$ ($\alpha \approx 1$ ns/elemento no geral, 0,5–3,0 ns por topologia, Tabela 8), a regra prática é $\Delta I \gtrsim 5n$ inversões removidas; os breakevens por topologia seguem da Tabela 8 (ex. 144.000 para Aleatório, 28.000 para Tubo em n=10.000). Cada desfecho em n=10.000 nas Tabelas 4–6 obedece a essa regra: Aleatório (ΔI=11,3M), Serra (12,5M) e Duplicados (10,3M) ganham, enquanto Tubo (10.000) e Runs (5.000) ficam abaixo do breakeven e são neutros.

3.5. Ameaças à Validade

Internas: máquina, SO (Windows 11), compilador (Rust 1.96.0, *release*, `opt-level=3`) e dados `i32` únicos; escalonamento de frequência da CPU, afinidade SMT e resolução do temporizador do Windows não foram controlados além do plano de alto desempenho e da minimização de processos em segundo plano. *De construção:* C_{pre} é medido por topologia dentro do mesmo harness Criterion (Tabela 8), de modo que a decomposição C_{pre} vs. ΔC_{sort} vale para a matriz relatada; apenas chaves `i32` e uma implementação por algoritmo foram testadas. *Externas:* nove topologias sintéticas mais um asset real de bytes, cinco *pools* independentes por célula, métodos quadráticos limitados a $n \leq 10^5$, Duplicados fixos em k=3 valores; incluem-se pontos até n=10^7 (≈40 MB por vetor `i32`) (Figura 4). *Estatísticas:* inferência pareada por *bootstrap* com estabilidade entre *pools* para as tabelas publicadas (n=10^4, n=10^6); demais tamanhos apoiam-se nos ICs do Criterion. *De medição:* sem contadores de

hardware (`perf_event_open` é específico do Linux); atribuições a preditor de desvios e cache são hipóteses, não medições. Todos os experimentos podem ser reproduzidos por meio do comando `cargo bench`, conforme descrito na Seção 7.

4. Resultados e Discussão

Esta seção apresenta a análise e discussão dos resultados experimentais obtidos, avaliando o impacto do pré-processamento simétrico sobre a estrutura dos dados e o tempo de execução dos algoritmos. A discussão parte do impacto direto na redução de inversões e da interpretação do viés geométrico observado no cenário invertido e avança para a mensuração do tempo total de ordenação em algoritmos quadráticos e quasilineares. O objetivo é demonstrar de forma consolidada os cenários em que a técnica oferece ganhos reais de desempenho e delimitar suas condições de aplicabilidade.

4.1. Redução de Inversões e Viés Estrutural das Entradas

A Figura 1 ilustra a variação das inversões observadas nos vetores de 10.000 elementos, cujos valores quantitativos exatos e porcentagens de redução estão detalhados na Tabela 3. Nas entradas não alinhadas com desordem não estruturada (Aleatório, Tartarugas, Quase ordenado, Duplicados, Real e Serra), o pré-processamento remove de 40% a 62% das inversões — uma redução absoluta $\Theta(n^2)$ de grande magnitude, integralmente medida neste trabalho nos vetores do *pool* pareado. A título de ordem de grandeza, esse patamar situa-se acima do termo de primeira ordem $n^2/12$ derivado como resultado próprio na Seção 3.4 para a componente de trocas espelhadas considerada isoladamente, o qual não se estende à passada completa; a hipótese dos autores é que o excedente esteja ligado à interação sequencial entre as trocas espelhadas e os ajustes adjacentes executados dentro da mesma passada, e não a uma contribuição aditiva dos ajustes, cada um dos quais remove no máximo uma inversão — hipótese não decomposta nem demonstrada em forma fechada neste artigo. Duas topologias não alinhadas resistem quase por completo à varredura: *Tubo* (0,04%) e *Runs* (0,16%), cuja geometria de desordem as comparações espelhadas jamais alcançam. Contudo, a interpretação da eliminação total das inversões (100%) no cenário *Invertido* exige sobriedade teórica e analítica.

[*] 99,99% (2.501 inversões restantes); [**] 100,00% exato (zero inversões restantes). A diferença de arredondamento impede interpretação de equivalência entre os dois casos. Tubo (0,04%) e Runs (0,16%) são estruturalmente invisíveis à varredura espelhada (Seção 4.1).

Como o Algoritmo 1 executa a troca direta entre $a[i]$ e $a[n-1-i]$, um vetor estritamente invertido (onde $a[i] > a[n-1-i]$ para todo $i < n/2$) sofre uma reversão geométrica completa da sua sequência em $O(n)$ operações. A resolução do caso Invertido decorre da geometria da transformação do pré-processamento, configurando um caso ideal onde a topologia da desordem coincide perfeitamente com a simetria aplicada.

O comportamento no cenário *Zigue-zague* é relatado como resultado empírico próprio da passada completa (redução a 2.501 inversões, 99,99%, Tabela 3), e não como consequência demonstrada das trocas espelhadas isoladas: este artigo não

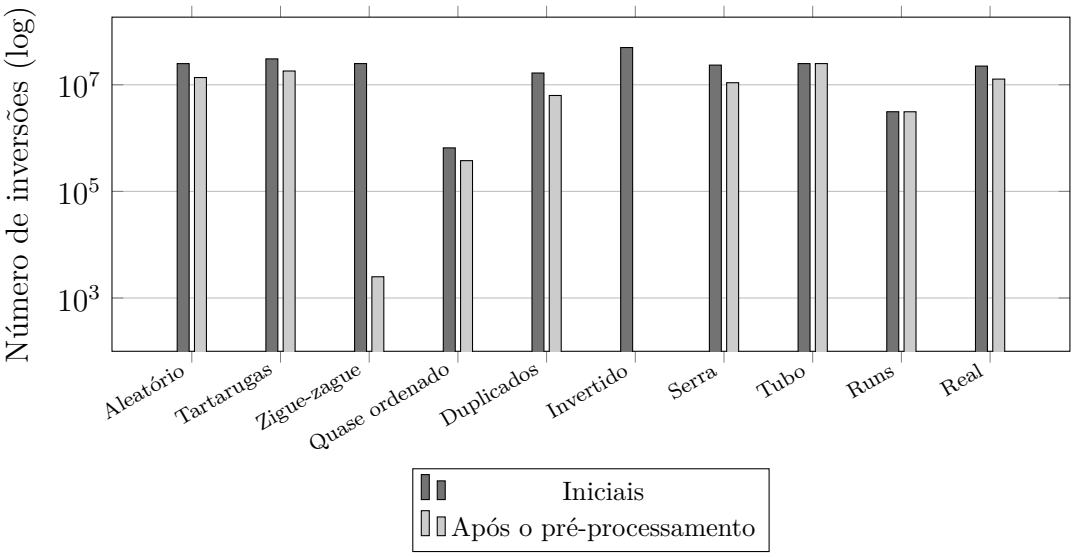


Figura 1. Número de inversões antes e após o pré-processamento simétrico para vetores de 10.000 elementos (escala logarítmica; no cenário Invertido o valor pós-pré é zero e não aparece na escala log).

Tabela 3. Redução de inversões proporcionada pelo pré-processamento simétrico em vetores de 10.000 elementos (média dos 50 vetores do *pool* pareado; mesma semente dos tempos da Tabela 10).

Tipo	Inversões	Pós-pré	Redução
Aleatório	24.960.120	13.681.366	45,19%
Tartarugas	30.613.750	18.133.750	40,77%
Zigue-zague	24.997.500	2.501	99,99%*
Quase ordenado	654.922	377.149	42,41%
Duplicados	16.633.993	6.300.346	62,12%
Invertido	49.995.000	0	100,00%**
Serra	23.400.000	10.937.472	53,26%
Tubo	24.995.000	24.985.004	0,04%
Runs	3.122.500	3.117.505	0,16%
Real	22.447.037	12.789.942	43,02%

reporta medição por componente isolado nessa topologia, de modo que nenhuma explicação baseada apenas na simetria das comparações espelhadas é reivindicada, e o caso permanece sem uma caracterização teórica fechada.

4.2. Impacto no Tempo de Execução e Custo do Pré-processamento

A Tabela 4 resume as faixas de ganho obtidas sob vetores de 10.000 elementos (valores por célula no Apêndice), evidenciando como a eficiência do pré-processamento depende da adaptabilidade de cada algoritmo. Em entradas não alinhadas, os métodos quadráticos adaptativos obtiveram ganhos expressivos: o *Insertion Sort* avançou de 28,6% a 60,3% (Aleatório $+43,9\%$) e o *Bubble Sort* de 14,9% a 26,2%. Os maiores benefícios concentraram-se nas entradas alinhadas à varredura: no cenário *Zigue-zague*, o tempo do *Insertion Sort* caiu de $5.249,97\,\mu s$ para $13,60\,\mu s$ ($\times 386$) e o do *Bubble Sort* de $18.469,42\,\mu s$ para $16,00\,\mu s$ ($\times 1.155$); no *Invertido*, os *speedups* alcançaram $\times 837$ e $\times 2.280$, respectivamente ($+99,9\%$ e $+100,0\%$).

Em contrapartida, algoritmos de complexidade $O(n \log n)$, como o *Merge Sort*, apresentaram resultados mistos em $n = 10.000$ (decomposição de $-14,2\%$ a $+15,7\%$; e até $+18,6\%$ em $n = 10^6$, Tabela 9), uma vez que o custo da etapa prévia tende a superar eventuais ganhos em distribuições sem estrutura favorável. Sob a validação por *bootstrap* pareado (Seção 3.3), não apresentaram diferença estatística significativa em $n = 10.000$ as combinações Merge/Duplicados ($-1,2\%$) e Selection para a maioria das entradas não alinhadas. Ademais, ressalta-se que Selection/Tartarugas ($+3,1\%$) mostrou-se instável entre *pools* e Quicksort/Quase ordenado ($-4,5\%$) não atingiu significância no pareamento.

Tabela 4. Faixa de ganho do pré-processamento por algoritmo para vetores de 10.000 elementos (mínimo e máximo sobre as seis topologias clássicas; tipos entre parênteses). Os valores por célula, com marcas de significância, estão na Tabela 10 do Apêndice.

Algoritmo	Ganho mínimo	Ganho máximo
Merge Sort	$-14,2\%$ (Tartarugas)	$+15,7\%$ (Zigue-zague)
Quicksort	$-38,4\%$ (Invertido)	$+0,5\%$ (Zigue-zague)
Insertion Sort	$+28,6\%$ (Quase ordenado)	$+99,9\%$ (Invertido)
Bubble Sort	$+14,9\%$ (Aleatório)	$+100,0\%$ (Invertido)
Selection Sort	$-9,1\%$ (Invertido)	$+3,1\%$ (Tartarugas)

O Insertion Sort com pré-processamento em entrada Aleatória ($2.921,09\,\mu s$) permanece $\approx$8,1$\times$ mais lento que o Quicksort puro ($360,74\,\mu s$), e o próprio `sort_unstable` ($80,41\,\mu s$) é $\approx$36$\times$ mais rápido que o melhor híbrido na mesma entrada — de modo que nenhum ganho contra o mesmo algoritmo se traduz em competitividade contra o ordenador padrão. No Invertido, porém, o `desc_reverse` puro ($11,03\,\mu s$) iguala o Insertion Sort com pré-processamento ($12,81\,\mu s$): a detecção trivial de run captura sozinha todo o ganho das entradas alinhadas, confirmando que os *speedups* extremos são propriedade da geometria da entrada, não do pré-processamento.

A Serra comporta-se como as topologias não estruturadas (Insertion $+52,3\%$, Bubble $+33,8\%$, `desc_reverse` $+51,7\%$), o Real acompanha de perto o Aleatório

Tabela 5. Baselines competitivos: tempo médio por execução (em µs, média ± meia-largura do IC 95%) e ganho do pré-processamento para vetores de 10.000 elementos. `sort_unstable` e `sort` são os ordenadores da biblioteca padrão do Rust; `desc_reverse` é o Insertion Sort precedido de detecção $O(n)$ de run descendente com reversão. Marcas de significância conforme a Seção 3.3.

Algoritmo	Tipo	Puro	Com pré	Ganho
`sort_unstable`	Aleatório	80,41 ± 2,35	107,55 ± 0,48	-33,8%†
`sort_unstable`	Tartarugas	47,26 ± 0,28	51,34 ± 0,28	-8,7%†
`sort_unstable`	Zigue-zague	70,63 ± 0,46	66,09 ± 0,29	+6,4%†
`sort_unstable`	Quase ordenado	76,10 ± 0,36	83,16 ± 0,43	-9,3%†
`sort_unstable`	Duplicados	15,08 ± 0,28	40,83 ± 0,19	-170,8%†
`sort_unstable`	Invertido	3,75 ± 0,12	7,86 ± 0,19	-109,7%†
`sort`	Aleatório	112,00 ± 0,69	140,99 ± 0,82	-25,9%†
`sort`	Tartarugas	23,44 ± 0,25	27,79 ± 0,17	-18,5%†
`sort`	Zigue-zague	93,45 ± 0,70	47,60 ± 0,29	+49,1%†
`sort`	Quase ordenado	124,33 ± 0,64	126,66 ± 0,47	-1,9%
`sort`	Duplicados	17,85 ± 0,55	43,79 ± 0,28	-145,3%†
`sort`	Invertido	4,48 ± 0,17	8,43 ± 0,22	-88,2%†
`desc_reverse`	Aleatório	5.297,50 ± 27,50	2.966,86 ± 17,17	+44,0%†
`desc_reverse`	Tartarugas	6.421,62 ± 30,47	3.826,81 ± 19,40	+40,4%†
`desc_reverse`	Zigue-zague	5.299,78 ± 31,93	13,42 ± 0,12	+99,8%†
`desc_reverse`	Quase ordenado	174,43 ± 3,28	126,71 ± 2,56	+27,4%†
`desc_reverse`	Duplicados	3.569,32 ± 17,29	1.414,40 ± 8,50	+60,4%†
`desc_reverse`	Invertido	11,03 ± 0,32	11,86 ± 0,21	-7,5%†

Tabela 6. Faixa de ganho do pré-processamento por método nas novas topologias para vetores de 10.000 elementos (mínimo e máximo sobre Serra, Tubo, Runs e Real). Os valores por célula, com marcas de significância, estão na Tabela 11 do Apêndice.

Método	Ganho mínimo	Ganho máximo
Merge Sort	$-5,2\%$ (Runs)	$+0,0\%$ (Real)
Quicksort	$-34,2\%$ (Serra)	$-1,0\%$ (Tubo)
Insertion Sort	$+0,0\%$ (Tubo)	$+52,3\%$ (Serra)
Bubble Sort	$-0,7\%$ (Tubo)	$+33,8\%$ (Serra)
Selection Sort	$-0,4\%$ (Serra)	$+1,2\%$ (Runs)
`sort_unstable`	$-26,8\%$ (Real)	$-6,3\%$ (Tubo)
`sort`	$-71,3\%$ (Tubo)	$-12,8\%$ (Serra)
`desc_reverse`	$-0,9\%$ (Tubo)	$+51,7\%$ (Serra)

(Insertion $+42,2\%$, Bubble $+13,6\%$), enquanto Tubo e Runs são neutros para os métodos clássicos ($-5,2\%$ a $+2,5\%$, majoritariamente não significativos) — consistente com sua redução de inversões de $\approx$0%. Os ordenadores padrão degradam sob pré-processamento nas novas topologias (até $-71,3\%$ para `sort` no Tubo), pois o custo $O(n)$ do pré-processamento não lhes traz benefício líquido.

A viabilidade prática dessa abordagem fundamenta-se no baixo custo computacional da etapa simétrica. Conforme demonstrado na Tabela 7 e na Figura 2, o tempo de execução do pré-processamento apresenta um comportamento estritamente linear $O(n)$ em relação ao tamanho do vetor, com tempo unitário entre $0,95$ e $1,15$ ns por elemento, mínimo em 10^6 elementos ($\approx$ 0,95 ns). Assim, a varredura impõe um custo adicional reduzido — apenas $11,51\,\mu s$ para 10.000 elementos —, amplamente compensado pela economia de milissegundos na ordenação adaptativa nos cenários em que há redução expressiva de inversões.

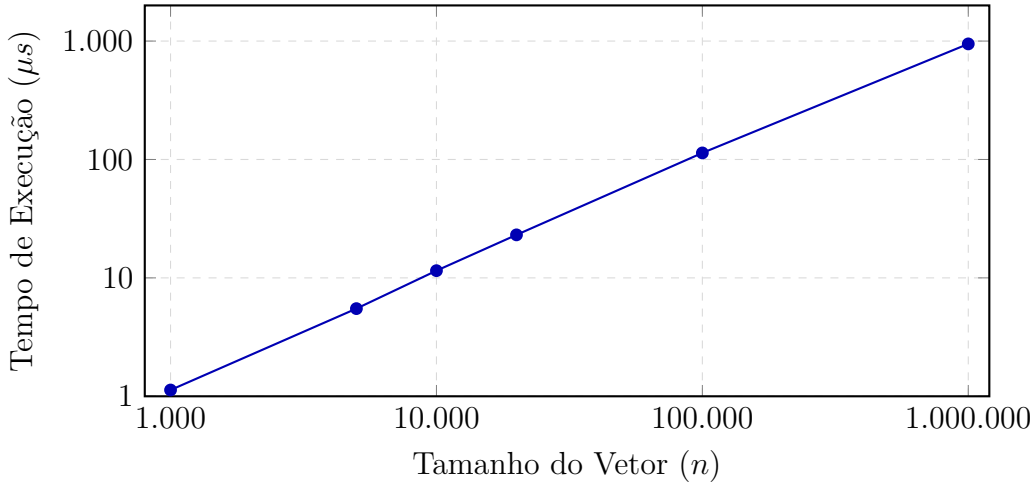


Figura 2. Tempo médio de execução do pré-processamento em função do tamanho do vetor (n), apresentado em escala log-log para evidenciar o comportamento linear $O(n)$. Valores médios sobre os dez tipos de entrada, obtidos pela ferramenta CLI com seed 42.

Tabela 7. Custo médio do pré-processamento simétrico em função do tamanho do vetor (média sobre os dez tipos; medido via ferramenta CLI, seed 42). O ponto n=20.000 foi incluído exclusivamente para caracterização da linearidade e não faz parte da matriz principal de tempos de ordenação.

Tamanho	Pré (µs)	ns/elemento
1.000	1,13	1,13
5.000	5,50	1,10
10.000	11,51	1,15
20.000	23,09	1,15
100.000	113,58	1,14
1.000.000	946,36	0,95

A heterogeneidade é em si informativa: o custo da varredura depende dos dados de entrada, de modo que entradas Aleatórias e com Duplicados apresentam custo

Tabela 8. Custo isolado do pré-processamento C_{pre} por topologia (mesmo harness Criterion da matriz de tempos; média, μs). A média CLI da Tabela 7 esconde forte heterogeneidade: entradas não estruturadas (Aleatório, Duplicados) custam ≈5× mais que as estruturadas em n=10.000. Real não tem ponto em 10^6 (teto do asset nativo).

Tipo	C_{pre} em 10^4	C_{pre} em 10^6
Aleatório	30,36	3.049,14
Tartarugas	5,66	537,28
Zigue-zague	5,72	545,30
Quase ordenado	7,31	713,73
Duplicados	27,63	2.668,85
Invertido	5,69	517,05
Serra	5,10	490,93
Tubo	5,86	589,98
Runs	5,49	558,90
Real	6,97	—

mais elevado (≈30 e 28 μs em n=10.000), enquanto entradas estruturadas custam ≈5–7 μs. Consequentemente, o *overhead* por célula deve ser lido na Tabela 8, não na média da Tabela 7: por exemplo, Merge/Aleatório em n=10.000 fica $+11,46\,\mu s$ mais lento contra um C_{pre} topologia-específico de $30,36\,\mu s$ — ou seja, a fase de ordenação em si melhorou ligeiramente e a degradação líquida decorre da aritmética entre o custo do pré-processamento e a economia na ordenação.

4.3. Tempo Total de Ordenação e Análise Individual dos Algoritmos

A avaliação do tempo total de execução revela uma dicotomia no impacto do pré-processamento. Nos algoritmos adaptativos de classe quadrática, a eliminação prévia das inversões proporciona ganhos expressivos quando a redução absoluta ΔI é expressiva ($14,9\%$ a $100,0\%$ em n=10.000), pois a varredura linear de baixo custo (≈ 1,ns/elemento) é amplamente compensada pelo trabalho local suprimido. Mecanisticamente, o tempo do *Insertion Sort* por inversão mostra-se notavelmente constante (≈ 0,21,ns/inversão em n=10.000 para a maioria das entradas), de modo que seus ganhos acompanham a redução de ΔI de forma direta, desviando apenas em entradas quase ordenadas (0,29,ns/inversão) pela dominância dos custos fixos sob pequeno volume de inversões.

Por outro lado, o tempo de execução do *Bubble Sort* permanece praticamente constante (16–23,ms) em entradas cujo volume de inversões varia por duas ordens de grandeza, revelando que seu desempenho é determinado pelo deslocamento máximo dos elementos à esquerda, e não por I. Esse comportamento explica seu ganho modesto em distribuições aleatórias ($+14,9\%$ apesar da queda de 45% nas inversões) em contraste com a redução para uma única passada em entradas alinhadas (*speedups* de ×1.155 a ×2.280). A avaliação nas novas topologias revalida essa dinâmica: a topologia *Serra* passa a exibir comportamento alinhado (ganhos de $+52,3\%$ no Insertion e $+33,8\%$ no Bubble), enquanto *Tubo* e *Runs* — onde a varredura não afeta a desordem (≈ 0% de redução) — mantêm o desempenho neutro ($-0,7\%$ a $+2,5\%$).

O *Merge Sort* exibe resultados mistos segundo o critério pareado. O ganho concentra-se nas topologias em que o pré-processamento elimina a desordem estrutural: *Zigue-zague* ($+15,7\%$) e *Invertido* ($+3,3\%$) em $n = 10.000$, crescendo para $+18,6\%$ e $+6,0\%$ em n=10^6 (Tabela 9). Em *Aleatório*, *Tartarugas* e *Quase ordenado*, porém, o total degrada de forma significativa. Essas degradações são aritméticas, como mostra a decomposição por topologia da Tabela 8, e não uma perturbação da distribuição de chaves.

O *Quicksort*, implementado com particionamento de três vias e mediana de três, apresenta degradação sistemática sob o pré-processamento. A perda é leve na maioria das topologias ($-4,5\%$ a $-8,0\%$ em *Aleatório*, *Tartarugas* e *Quase ordenado*; *Zigue-zague* fica em $+0,5\%$), mas torna-se acentuada em *Duplicados* ($-34,4\%$) e *Invertido* ($-38,4\%$). Novamente a escala do *overhead* importa: Quicksort/Duplicados fica $+14,25\,\mu s$ mais lento em n=10.000 (C_{pre} topologia-específico de $27,63\,\mu s$) e $+1,24$ ms em n=10^6 (C_{pre} de $2,67$ ms), de modo que a maior parte da degradação é o próprio custo do pré-processamento, não uma perturbação da distribuição de chaves.

No cenário *Invertido*, a entrada ordenada produzida pelo pré-processamento é ordenada mais devagar pela implementação avaliada do que o vetor original: 328,39 μs vs. 454,47 μs em n=10^4 e 267,65 ms vs. 368,64 ms em n=10^6 (Tabelas 10 e 12), enquanto no Aleatório os tempos são 360,74 μs vs. 378,06 μs e 53,27 ms vs. 54,67 ms nas mesmas escalas. O crescimento entre as duas escalas é acentuadamente maior no braço ordenado do que no aleatório, e o cenário Tubo (*OrganPipe*) exibe o crescimento mais extremo (295,87 μs em n=10^4 para 4.070,08 ms em n=10^6 no braço puro, Tabelas 11 e 12). Nenhuma contagem interna (partições, comparações, profundidade de pilha/recursão) é reportada neste artigo: o módulo instrumentado (`quick_stats`, exemplo `diag_f2`) é cópia fora da matriz e não fundamenta as tabelas. A compatibilidade desses tempos com a implementação (mediana de três entre início, meio e fim com particionamento de três vias, verificada em `src/core/sort/quick.rs`) foi conferida apenas por inspeção e por reprodução qualitativa da degeneração em simulação avulsa; a implementação não foi validada contra referência externa, de modo que não se distingue aqui entre característica do esquema mediana-de-três/três-vias e defeito de codificação, e nenhum mecanismo interno (partições desbalanceadas, DNF, preditor/cache) é reivindicado como causa. A validação contra implementação de referência (pdqsort) permanece como trabalho futuro.

Assim, para a classe $O(n \log n)$, a técnica não se mostra benéfica na implementação avaliada, com exceções pontuais (Merge/Zigue-zague e Merge/Invertido em ambas as escalas). Quicksort/Quase ordenado, a única exceção aparente em larga escala ($+5,5\%$ em n=10^6, após $-4,5\%$ em n=10.000 e $-9,3\%$ em n=10^5), troca de sinal entre *pools* na análise pareada e é, portanto, relatado como não robusto em vez de ganho. Esses resultados delimitam a aplicabilidade do pré-processamento, indicando que a redução de inversões, isoladamente, não garante ganhos para algoritmos dessa classe.

O comportamento do *Selection Sort* confirma sua natureza não adaptativa: como o número de comparações é fixo ($n(n-1)/2$) independentemente da configuração inicial, a redução das inversões não se traduz em ganho algorítmico. De fato, nas medições o algoritmo permaneceu não significativo na maioria das topologias

sob o critério pareado (variações de $-1,0\%$ a $+3,1\%$; Aleatório, Quase ordenado, Duplicados e Zigue-zague não significativos, Tartarugas $+3,1\%$ significativo na seed 42 mas instável entre *pools*), com exceção de uma degradação consistente no cenário *Invertido* em $n = 10.000$ ($19.150,38 \pm 353,49\,\mu s$ vs. $20.900,45 \pm 113,30\,\mu s$), também reproduzida em $n = 100.000$ ($-12,8\%$, $1.849,65 \pm 8,34$ ms vs. $2.086,10 \pm 8,82$ ms). Nas novas topologias (Serra, Tubo, Runs, Real), todas as variações ficam entre $-0,4\%$ e $+1,2\%$ e não significativas — uma confirmação limpa de neutralidade.

No cenário *Invertido*, o pré-processamento transforma a entrada em uma sequência ordenada e observa-se uma degradação consistente de 9% a 13% ($19.150,38 \pm 353,49\,\mu s$ vs. $20.900,45 \pm 113,30\,\mu s$ em n=10.000; $-12,8\%$ em n=100.000). A causa exata dessa degradação não foi investigada neste trabalho (sem contadores de hardware, Seção 3.5). De todo modo, a magnitude dessa degradação é modesta ($\leq 13\%$) e sem ganho sistemático, o que é consistente com a premissa teórica de que algoritmos não adaptativos não exploram a redução de inversões [Estivill-Castro and Wood 1992].

A Figura 3 sintetiza essa dinâmica nas dez topologias: aproveitamento expressivo nos métodos quadráticos adaptativos, resposta mista do *Merge Sort*, degradação sistemática do *Quicksort* de três vias e neutralidade do *Selection Sort* não adaptativo (com a exceção do cenário Invertido). A Tabela 9 complementa a análise para n=10^6 nos métodos quasilineares.

Tabela 9. Faixa de ganho do pré-processamento por método para vetores de 1.000.000 **de elementos (mínimo e máximo sobre as topologias medidas; Real excluído pelo teto do asset nativo). Os valores por célula, com marcas de significância, estão na Tabela 12 do Apêndice.**

Método	Ganho mínimo	Ganho máximo
Merge Sort	$-11,5\%$ (Tartarugas)	$+18,6\%$ (Zigue-zague)
Quicksort	$-37,7\%$ (Invertido)	$+5,5\%$ (Quase ordenado)
`sort_unstable`	$-194,3\%$ (Duplicados)	$+20,0\%$ (Zigue-zague)
`sort`	$-98,0\%$ (Duplicados)	$+57,5\%$ (Zigue-zague)

Os valores de n=10^6 replicam o padrão de n=10.000: ganhos do Merge Sort restritos a Zigue-zague ($+18,6\%$), Invertido ($+6,0\%$) e Duplicados ($+4,8\%$, significativo na seed 42 mas instável entre *pools*), degradação sistemática do Quicksort (até $-37,7\%$), com Tartarugas ($+0,0\%$) e Quase ordenado ($+5,5\%$) reprovados no teste pareado de robustez, confirmando que os ganhos consistentes em múltiplas topologias se concentram nos métodos adaptativos, com as exceções isoladas do Merge/Zigue-zague e do Merge/Invertido relatadas acima. Sob o critério pareado, as células não significativas em n=10^6 são Merge/Aleatório ($-0,9\%$) e Quicksort/Quase ordenado.

A Figura 4 estende o quadro por todos os tamanhos coletados (n=16–10^7): Insertion/Aleatório sobe de ≈0% em n=16 a um platô estável de ≈44% desde n=10^4; Bubble/Aleatório é não monotônico em n pequeno (pico de $+31,9\%$ em n=64) com alta em 10^5 (8,9–25,6%); Merge/Zigue-zague cresce monotonicamente (1,8–18,6%), enquanto Merge/Invertido é negativo em n pequeno (até $-5,3\%$), positivo no meio ($+3,3$–$+11,5\%$) e passa a $-2,2\%$ em 10^7; Quicksort/Aleatório atenua com o tamanho

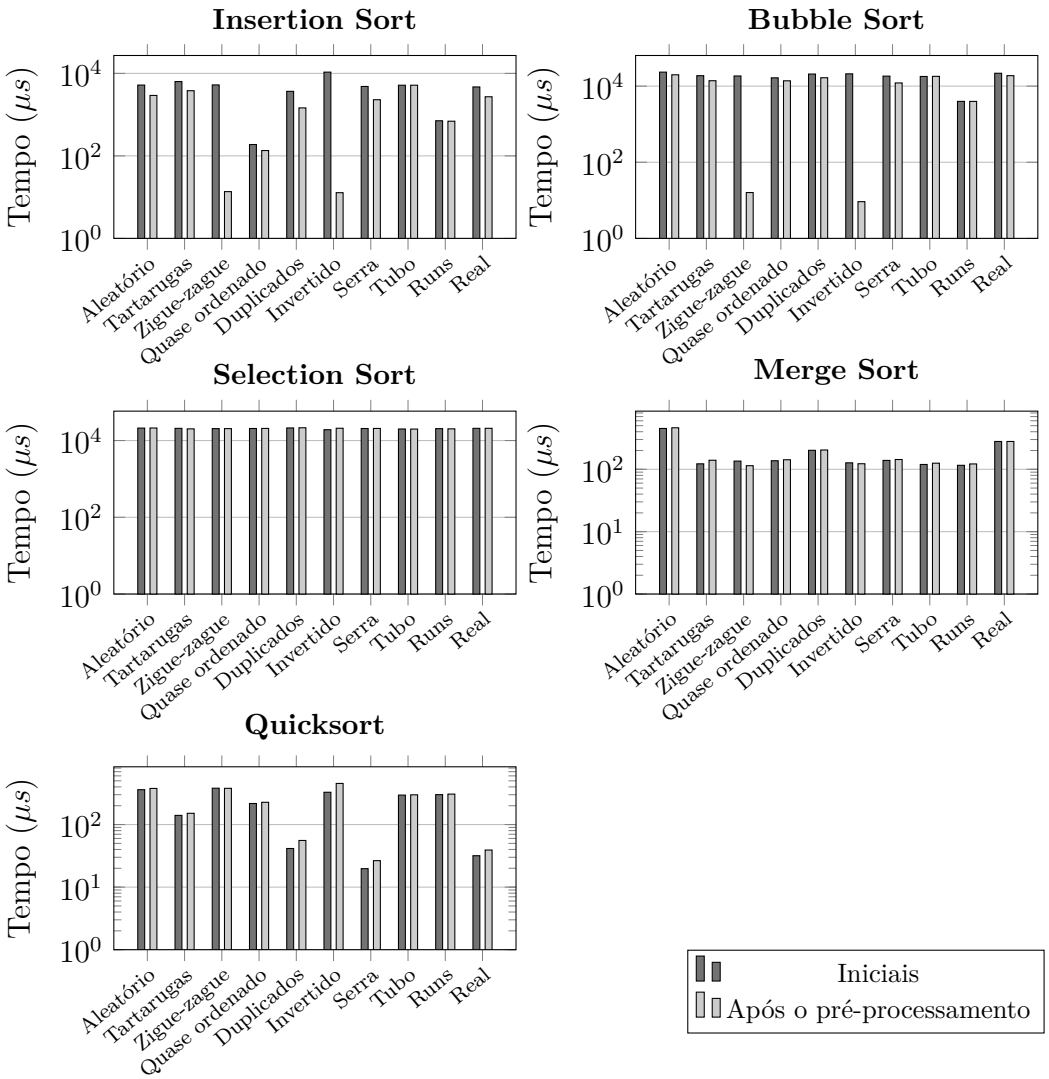


Figura 3. Comparação temporal entre a ordenação pura e com pré-processamento simétrico ($n = 10.000$) nas dez topologias, em escala logarítmica.

($-10,6$ a $-1,8\%$); Quicksort/Quase ordenado é não monotônico e positivo apenas em 10^6 ($+5,5\%$, não robusto entre *pools*). Contra a referência `sort_unstable`, o método é sistematicamente desfavorável em todos os tamanhos (StdUnstable/Aleatório de $-0,9\%$ em n=16 até $-77,5\%$ em n=1.000), e Insertion/Serra só fica positivo acima de n=128 ($+17$–$+53\%$), com Insertion/Tubo plano próximo de zero — evidência de que as tendências por tamanho não se estabilizam uniformemente e devem ser relatadas por topologia.

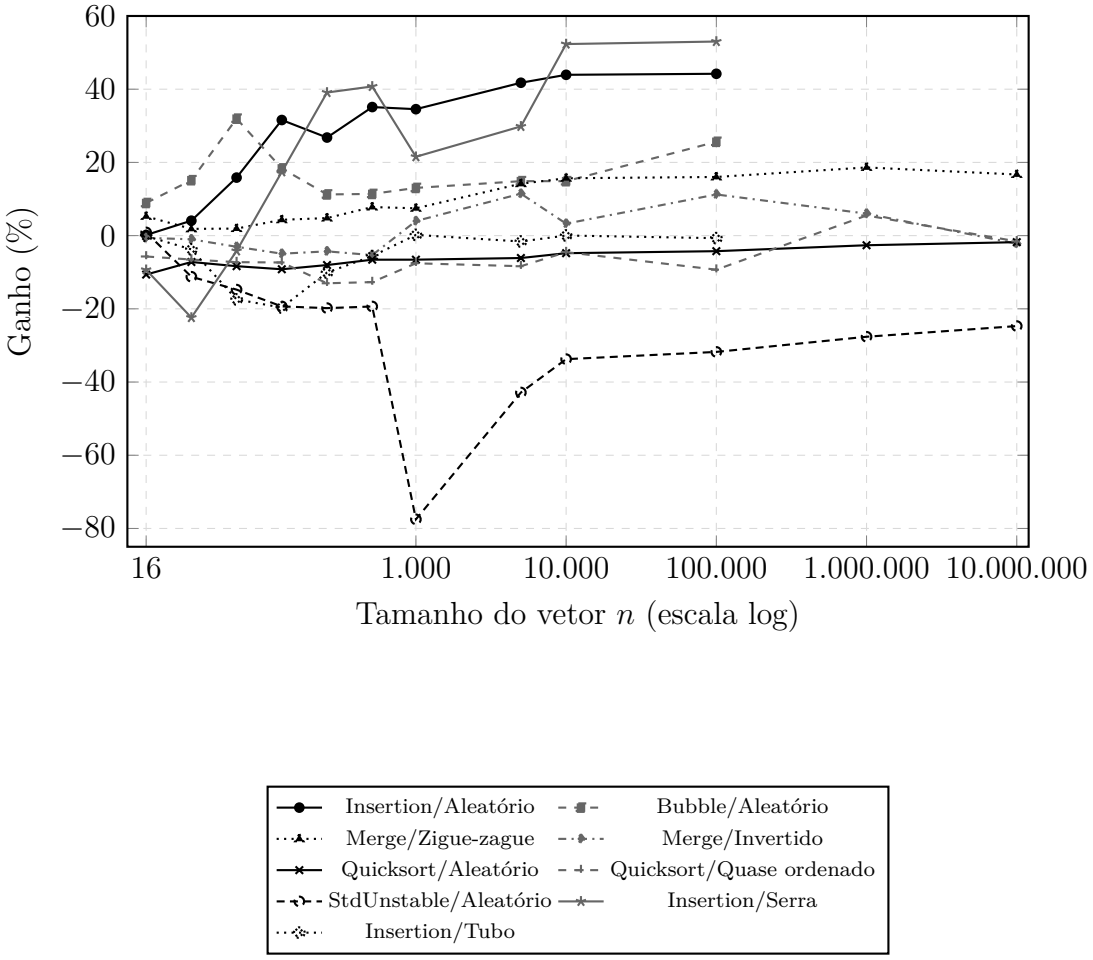


Figura 4. Ganho do pré-processamento em função do tamanho da entrada para pares algoritmo–topologia selecionados (mesmas médias do Criterion; métodos quadráticos limitados a n=10^5; lacunas são células não planejadas).

5. Considerações Finais

Este trabalho apresentou e avaliou um pré-processamento simétrico de complexidade $O(n)$ destinado à redução do número de inversões em sequências de entrada para algoritmos de ordenação. Na maioria das entradas não alinhadas, a abordagem removeu de 40% a 62% das inversões a ≈1,0 ns/elemento, gerando ganhos de 14% a 60% em algoritmos quadráticos adaptativos (Insertion Sort e Bubble Sort) em n=10.000; a eliminação quase total de inversões e os *speedups* de ×386–2.280 ocorreram apenas no Invertido (reversão geométrica pela simetria da varredura) e no Zigue-zague (efeito empírico da passada completa, sem caracterização teórica fechada) e não devem ser

generalizados. Os ganhos se materializam quando a redução do custo de ordenação supera o custo da etapa preliminar, satisfazendo $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$; para métodos adaptativos, isso assume a forma acionável $\Delta I \gtrsim 5n$ (Seção 3.4), obedecida por todos os desfechos em n=10.000. As comparações contra o mesmo algoritmo são relatadas ao lado dos ordenadores mais rápidos disponíveis (`sort_unstable`/`sort`) e da detecção de runs, que delimitam o escopo prático (Seção 4.2): em entradas aleatórias o melhor híbrido permanece muito mais lento que o ordenador padrão, e no Invertido a detecção de runs sozinha iguala o desempenho do híbrido, de modo que os *speedups* extremos nas entradas alinhadas não configuram vantagem geral sobre as alternativas padrão.

Em relação ao algoritmo quadrático não adaptativo avaliado, o Selection Sort confirmou a hipótese de neutralidade: variações de $-1,0\%$ a $+3,1\%$, robustamente significativas apenas no Invertido sob a regra pareada (Tartarugas significativo na seed 42 mas instável entre *pools*, Zigue-zague não significativo), com exceção de uma degradação consistente de ≈9–13% na entrada totalmente invertida (ICs pareados de 95%: $-9,1\%$ em n=10.000, $-12,8\%$ em n=100.000), cuja causa exata não foi investigada neste trabalho (Seção 4.3). Esse resultado, previsto teoricamente pela invariância do número de comparações ($n(n-1)/2$), delimita claramente que a redução de inversões não se converte em ganho para métodos não adaptativos.

Por outro lado, nos algoritmos de complexidade $O(n \log n)$ avaliados, a aplicação do método revelou limitações importantes. Com a implementação de Quicksort utilizada (particionamento de três vias e mediana de três, `src/core/sort/quick.rs`, sem validação contra referência externa), o pré-processamento causou degradação sistemática de desempenho — de 4% a 8% na maioria das topologias ($-4,5\%$ para Zigue-zague em n=10^6) e de ≈31%–38% em *Duplicados* e *Invertido*. Em *Duplicados* a degradação cabe na escala do próprio *overhead* de pré-processamento; no *Invertido* e no *Tubo*, ao contrário, a entrada ordenada/estruturada resultante é ordenada muito mais devagar pela implementação avaliada (Seção 4.3), sem que nenhuma causa interna seja reivindicada neste artigo.

No Merge Sort os efeitos foram mistos e dependentes da topologia: ganhos apenas nas topologias alinhadas à varredura simétrica (Zigue-zague $+15,7\%$ em n=10.000 e $+18,6\%$ em n=10^6; Invertido $+3,3\%$ e $+6,0\%$, respectivamente, Tabelas 10 e 12) e degradação nas demais topologias clássicas e nas novas topologias. Esses ganhos isolados não se mantêm em todas as escalas: no Invertido, o ganho passa a $-2,2\%$ em n=10^7 (Figura 4), enquanto o Zigue-zague permanece positivo ($+16,7\%$ em n=10^7). Padrão análogo, restrito ao Zigue-zague, foi observado nos ordenadores da biblioteca padrão: `sort` ($+49,1\%$ em n=10.000 e $+57,5\%$ em n=10^6) e `sort_unstable` ($+6,4\%$ e $+20,0\%$, respectivamente), com degradação sistemática em todas as demais topologias, incluindo o Aleatório (`sort` $-25,9\%$ e $-19,4\%$; `sort_unstable` $-33,8\%$ e $-27,6\%$, em n=10.000 e n=10^6) e as novas topologias (Tabela 6). Assim, os ganhos mais expressivos e consistentes permaneceram nos algoritmos quadráticos adaptativos (Insertion Sort e Bubble Sort), os únicos a melhorar em múltiplas topologias não alinhadas além das alinhadas, enquanto os ganhos em métodos quasilineares e nos ordenadores padrão foram isolados a topologia(s) alinhada(s) à varredura e não se estenderam a entradas aleatórias ou não alinhadas;

em particular, nenhum benefício sistemático foi observado para o algoritmo não adaptativo avaliado (Selection Sort).

Como limitações, registram-se a restrição a uma única máquina com Windows e a chaves `i32`, o controle apenas parcial de escalonamento de frequência, afinidade SMT e resolução do temporizador; o viés de seleção das topologias sintéticas e o caráter operacional do rótulo "Real"; a inferência pareada com estabilidade entre *pools* restrita a $n{=}10^4$ e $n{=}10^6$; e a ausência de contadores de hardware, de modo que as atribuições a predição de desvios e cache permanecem hipóteses (Seção 3.5).

Como trabalhos futuros, recomenda-se (i) a derivação de uma forma fechada para a passada completa, incluindo a interação entre trocas espelhadas e ajustes adjacentes (o termo de primeira ordem $\approx n^2/12$ da Seção 3.4 é derivação própria deste trabalho e vale somente para a componente espelhada isolada, sem atribuição a Hwang et al.), bem como uma caracterização teórica do caso Zigue-zague, relatado aqui apenas como resultado empírico próprio, e a validação da anomalia do Quicksort contra uma implementação de referência (pdqsort); (ii) a realização de testes em segunda máquina e em ambiente Linux com a infraestrutura `perf_event`, viabilizando a medição direta de *cache misses* e *branch mispredictions*; e (iii) a exploração de particionamentos alternativos, passadas tipo Shell e técnicas adaptativas recentes como *BlockQuicksort* [Edelkamp and Weiß 2016] e *Timsort* [Auger et al. 2018] como base comparativa.

6. Uso de Inteligência Artificial

Durante o desenvolvimento deste trabalho, os autores utilizaram ferramentas de inteligência artificial generativa como apoio à elaboração e à revisão do texto, à conferência da execução dos testes e à organização da análise dos resultados experimentais. Essas ferramentas não foram utilizadas para gerar dados experimentais ou produzir resultados numéricos. Todo conteúdo, decisão metodológica, resultado e interpretação apresentados neste artigo foram conferidos, discutidos e validados pelos autores, que mantêm responsabilidade integral pela precisão, originalidade e integridade do trabalho. O uso de IA não substituiu o julgamento científico dos autores nem a responsabilidade destes sobre o conteúdo científico.

7. Repositório da Pesquisa

O código-fonte em Rust, o *harness* de *benchmark*, os scripts de geração e de análise dos dados, o *asset* de bytes utilizado na topologia Real e os arquivos CSV que fundamentam as tabelas e figuras deste artigo estão disponíveis publicamente no repositório `https://github.com/Zidan-09/presorting-algorithm`. O repositório reúne a implementação dos algoritmos de ordenação avaliados, do pré-processamento simétrico e os procedimentos experimentais necessários à reprodução dos resultados, que pode ser realizada por meio do comando `cargo bench`.

A. Tabelas completas de tempos de execução

Este apêndice apresenta os valores por célula (média ± meia-largura do IC 95%) que fundamentam as Tabelas 4, 6 e 9, com as marcas de significância do delineamento pareado (* IC 95% do *bootstrap* exclui o zero na seed 42; † adicionalmente estável em sinal nos cinco *pools*).

Tabela 10. Tempo médio por execução (em µs) e ganho do pré-processamento para vetores de 10.000 elementos: algoritmos clássicos nas seis topologias clássicas. Fonte: Criterion, seed 42.

Algoritmo	Tipo	Puro	Com pré	Ganho
Merge Sort	Aleatório	451,57 ± 1,66	463,03 ± 1,49	-2,5%†
Merge Sort	Tartarugas	122,36 ± 0,60	139,68 ± 1,82	-14,2%†
Merge Sort	Zigue-zague	135,37 ± 0,66	114,16 ± 0,51	+15,7%†
Merge Sort	Quase ordenado	137,14 ± 1,58	142,22 ± 0,93	-3,7%†
Merge Sort	Duplicados	201,45 ± 0,73	203,87 ± 1,66	-1,2%
Merge Sort	Invertido	126,92 ± 0,96	122,77 ± 1,09	+3,3%†
Quicksort	Aleatório	360,74 ± 1,18	378,06 ± 1,34	-4,8%*
Quicksort	Tartarugas	140,18 ± 1,75	151,37 ± 1,47	-8,0%*
Quicksort	Zigue-zague	381,21 ± 4,46	379,51 ± 4,31	+0,5%*
Quicksort	Quase ordenado	217,15 ± 1,39	226,97 ± 1,35	-4,5%
Quicksort	Duplicados	41,45 ± 0,35	55,70 ± 0,56	-34,4%†
Quicksort	Invertido	328,39 ± 4,70	454,47 ± 9,17	-38,4%†
Insertion Sort	Aleatório	5.209,69 ± 40,87	2.921,09 ± 18,70	+43,9%†
Insertion Sort	Tartarugas	6.330,42 ± 40,35	3.801,57 ± 15,96	+40,0%†
Insertion Sort	Zigue-zague	5.249,97 ± 31,16	13,60 ± 0,16	+99,7%†
Insertion Sort	Quase ordenado	188,55 ± 4,27	134,69 ± 3,60	+28,6%†
Insertion Sort	Duplicados	3.673,33 ± 32,52	1.456,81 ± 19,69	+60,3%†
Insertion Sort	Invertido	10.725,42 ± 98,39	12,81 ± 0,23	+99,9%†
Bubble Sort	Aleatório	23.314,17 ± 91,56	19.847,68 ± 62,53	+14,9%†
Bubble Sort	Tartarugas	18.790,39 ± 62,61	13.857,28 ± 72,47	+26,2%†
Bubble Sort	Zigue-zague	18.469,42 ± 67,66	16,00 ± 0,18	+99,9%†
Bubble Sort	Quase ordenado	16.513,40 ± 137,86	13.788,51 ± 51,03	+16,5%†
Bubble Sort	Duplicados	20.765,80 ± 84,42	16.600,83 ± 97,35	+20,1%†
Bubble Sort	Invertido	21.031,38 ± 140,17	9,22 ± 0,19	+100,0%†
Selection Sort	Aleatório	21.106,62 ± 176,37	21.226,87 ± 113,10	-0,6%
Selection Sort	Tartarugas	20.797,92 ± 86,62	20.156,05 ± 106,47	+3,1%*
Selection Sort	Zigue-zague	20.470,58 ± 52,68	20.447,90 ± 62,51	+0,1%
Selection Sort	Quase ordenado	20.617,08 ± 87,96	20.704,93 ± 119,86	-0,4%
Selection Sort	Duplicados	21.299,72 ± 150,99	21.516,23 ± 169,19	-1,0%
Selection Sort	Invertido	19.150,38 ± 353,49	20.900,45 ± 113,30	-9,1%†

Tabela 11. Tempo médio por execução (em µs) e ganho do pré-processamento para vetores de 10.000 elementos: todos os oito métodos nas novas topologias.

Algoritmo	Tipo	Puro	Com pré	Ganho
Merge Sort	Serra	138,85 ± 0,69	143,46 ± 0,70	-3,3%*
Merge Sort	Tubo	119,53 ± 0,66	125,29 ± 0,81	-4,8%†
Merge Sort	Runs	115,81 ± 0,54	121,84 ± 0,41	-5,2%†
Merge Sort	Real	279,28 ± 1,11	279,26 ± 1,20	+0,0%*
Quicksort	Serra	19,74 ± 0,19	26,48 ± 0,20	-34,2%†
Quicksort	Tubo	295,87 ± 2,20	298,74 ± 2,06	-1,0%†
Quicksort	Runs	300,73 ± 2,62	306,88 ± 2,83	-2,0%
Quicksort	Real	31,78 ± 0,21	39,17 ± 0,26	-23,3%†
Insertion Sort	Serra	4.820,54 ± 21,14	2.297,88 ± 13,80	+52,3%†
Insertion Sort	Tubo	5.168,19 ± 25,43	5.166,41 ± 29,37	+0,0%
Insertion Sort	Runs	710,33 ± 10,82	692,84 ± 4,64	+2,5%
Insertion Sort	Real	4.691,35 ± 37,82	2.710,84 ± 14,21	+42,2%†
Bubble Sort	Serra	18.316,55 ± 110,17	12.120,68 ± 67,03	+33,8%†
Bubble Sort	Tubo	17.928,72 ± 51,51	18.059,14 ± 49,91	-0,7%
Bubble Sort	Runs	3.977,62 ± 21,96	3.965,26 ± 16,84	+0,3%*
Bubble Sort	Real	21.779,73 ± 66,43	18.826,62 ± 63,05	+13,6%†
Selection Sort	Serra	20.595,88 ± 66,65	20.682,48 ± 82,46	-0,4%
Selection Sort	Tubo	20.004,16 ± 71,15	19.909,36 ± 65,78	+0,5%
Selection Sort	Runs	20.419,03 ± 55,93	20.175,78 ± 55,75	+1,2%
Selection Sort	Real	20.810,97 ± 72,65	20.823,25 ± 68,90	-0,1%
`sort_unstable`	Serra	23,16 ± 0,22	28,98 ± 0,14	-25,1%†
`sort_unstable`	Tubo	71,66 ± 0,33	76,19 ± 0,46	-6,3%†
`sort_unstable`	Runs	61,31 ± 0,28	66,90 ± 0,35	-9,1%†
`sort_unstable`	Real	29,29 ± 0,21	37,13 ± 0,26	-26,8%†
`sort`	Serra	28,87 ± 0,40	32,55 ± 0,21	-12,8%†
`sort`	Tubo	20,00 ± 0,13	34,25 ± 0,14	-71,3%†
`sort`	Runs	29,62 ± 0,21	38,06 ± 0,19	-28,5%†
`sort`	Real	34,01 ± 0,41	39,27 ± 0,29	-15,5%†
`desc_reverse`	Serra	4.858,18 ± 36,03	2.346,63 ± 33,87	+51,7%†
`desc_reverse`	Tubo	5.172,80 ± 33,30	5.221,97 ± 31,13	-0,9%
`desc_reverse`	Runs	695,96 ± 4,87	698,58 ± 5,46	-0,4%
`desc_reverse`	Real	4.690,61 ± 28,75	2.720,19 ± 21,05	+42,0%†

Tabela 12. Tempo médio (ms) e ganho do pré-processamento para vetores de 1.000.000 de elementos: métodos quasilineares e baselines padrão (quadráticos omitidos por inviabilidade temporal). Fonte: Criterion, 50 amostras, IC 95%. Linhas abaixo da regra são as novas topologias (Real excluído pelo teto do asset nativo).

Algoritmo	Tipo	Puro (ms)	Com pré (ms)	Ganho
Merge Sort	Aleatório	68,44 ± 0,80	69,04 ± 0,59	-0,9%
Merge Sort	Tartarugas	16,66 ± 0,16	18,57 ± 0,38	-11,5%†
Merge Sort	Zigue-zague	18,62 ± 0,30	15,15 ± 0,14	+18,6%†
Merge Sort	Quase ordenado	20,15 ± 0,30	20,57 ± 0,28	-2,1%*
Merge Sort	Duplicados	26,57 ± 0,33	25,29 ± 0,27	+4,8%*
Merge Sort	Invertido	15,98 ± 0,32	15,02 ± 0,13	+6,0%†
Quicksort	Aleatório	53,27 ± 0,14	54,67 ± 0,11	-2,6%†
Quicksort	Tartarugas	101,17 ± 0,75	101,17 ± 0,77	+0,0%*
Quicksort	Zigue-zague	314,61 ± 2,36	328,79 ± 2,27	-4,5%†
Quicksort	Quase ordenado	32,74 ± 0,30	30,94 ± 0,26	+5,5%
Quicksort	Duplicados	3,97 ± 0,02	5,21 ± 0,02	-31,4%†
Quicksort	Invertido	267,65 ± 1,56	368,64 ± 1,04	-37,7%†
`sort_unstable`	Aleatório	10,81 ± 0,07	13,80 ± 0,06	-27,6%†
`sort_unstable`	Tartarugas	6,77 ± 0,02	7,02 ± 0,03	-3,6%*
`sort_unstable`	Zigue-zague	12,07 ± 0,14	9,66 ± 0,04	+20,0%†
`sort_unstable`	Quase ordenado	10,99 ± 0,03	11,51 ± 0,03	-4,8%†
`sort_unstable`	Duplicados	1,39 ± 0,01	4,10 ± 0,02	-194,3%†
`sort_unstable`	Invertido	0,40 ± 0,01	0,76 ± 0,01	-87,1%†
`sort`	Aleatório	15,10 ± 0,04	18,02 ± 0,10	-19,4%†
`sort`	Tartarugas	3,02 ± 0,02	3,12 ± 0,02	-3,4%*
`sort`	Zigue-zague	15,54 ± 0,05	6,60 ± 0,03	+57,5%†
`sort`	Quase ordenado	11,60 ± 0,16	12,06 ± 0,04	-4,0%†
`sort`	Duplicados	2,53 ± 0,02	5,01 ± 0,03	-98,0%†
`sort`	Invertido	0,42 ± 0,00	0,79 ± 0,01	-88,6%†
Merge Sort	Serra	18,86 ± 0,15	19,29 ± 0,20	-2,3%
Merge Sort	Tubo	15,56 ± 0,20	16,01 ± 0,15	-2,9%
Merge Sort	Runs	14,90 ± 0,26	15,56 ± 0,21	-4,4%
Quicksort	Serra	2,94 ± 0,01	3,22 ± 0,01	-9,3%†
Quicksort	Tubo	4.070,08 ± 9,70	4.071,58 ± 10,51	+0,0%*
Quicksort	Runs	248,60 ± 0,97	248,97 ± 0,94	-0,1%
`sort_unstable`	Serra	2,52 ± 0,01	2,77 ± 0,01	-10,2%*
`sort_unstable`	Tubo	11,98 ± 0,03	12,53 ± 0,03	-4,6%*
`sort_unstable`	Runs	9,35 ± 0,02	9,82 ± 0,02	-5,0%†
`sort`	Serra	3,53 ± 0,01	3,53 ± 0,01	+0,0%
`sort`	Tubo	2,45 ± 0,06	3,88 ± 0,05	-58,1%†
`sort`	Runs	3,25 ± 0,01	4,00 ± 0,01	-22,9%†

Referências

Auger, N., Jugé, V., Nicaud, C., and Pivoteau, C. (2018). On the worst-case complexity of timsort. In *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, volume 112 of *LIPIcs*, pages 4:1–4:13.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, 4th edition.

Edelkamp, S. and Weiß, A. (2016). Blockquicksort: Avoiding branch mispredictions in quicksort. In *Proceedings of the 24th Annual European Symposium on Algorithms (ESA 2016)*, volume 57 of *LIPIcs*, pages 38:1–38:16.

Estivill-Castro, V. and Wood, D. (1992). A survey of adaptive sorting algorithms. *ACM Computing Surveys*, 24(4):441–476.

Gil, A. C. (2019). *Métodos e técnicas de pesquisa social*. Editora Atlas Ltda.

Hwang, H.-K., Yang, B.-Y., and Yeh, Y.-N. (2000). Presorting algorithms: An average-case point of view. *Theoretical Computer Science*, 242(1-2):29–40.

Knuth, D. E. (1973). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA.

Mannila, H. (1985). Measures of presortedness and optimal sorting algorithms. *IEEE Transactions on Computers*, C-34(4):318–325. Versão estendida publicada originalmente como resumo no ICALP 1984 (LNCS 172, pp. 324–336).

Sampieri, R. H., Collado, C. F., and Lucio, M. d. P. B. (2021). *Metodologia de pesquisa*. Penso.

Sedgewick, R. and Wayne, K. D. (2011). *Algorithms*. Addison-Wesley, Upper Saddle River, 4th edition.

Ziviani, N. (2021). *Projeto De Algoritmos Com Implementações Em Java E C++*. Cengage Learning.